情報処理安全確保支援士試験　科目A-2　中分類25 情報セキュリティ対策　#11

ID連携とシングルサインオン

一度のログインで、たくさんのサービスへ。パスワードを渡さずに、認証の結果と権限を渡す

各ページの下に、根拠にした資料（IPA の支援士試験シラバス案、RFC、OASIS の SAML 2.0、OpenID Connect Core 1.0 など）を書いた。過去問は IPA 公開の午前 II 。
例の場面は Claude が作った

全体像 シラバス案 中分類25 小分類4「利用者の認証・認可・ID 連携」

1 SSO と認証基盤
リバースプロキシ・エージェント・代理認証、Kerberos（KDC・TGT）、LDAP、IDaaS、SCIM

2 SAML による ID 連携
IdP・SP・アサーション・XML 署名、SP-initiated

3 OAuth 2.0 と OpenID Connect
認可サーバ・アクセストークン・リフレッシュトークン・PKCE・ID トークン・JWT

過去の午前 II での出方（13回出題）

テーマ	出た回
OASIS が策定したのは（SAML）	H31春・R4秋・R6春
SAML 認証の特徴・流れ	R3秋・R5春・R7春
SAML の説明	R4春・R5秋
SSO の方式	R4秋・R6秋
OAuth 2.0 は認可か認証か	R5秋・R7春
OAuth 2.0 でトークンを発行するのは	R6秋

出典：IPA「支援士試験 シラバス（案）Ver.0.1」中分類25 小分類4／IPA 午前 II 各回（このPDFの問題演習を参照）

第1章

シングルサインオンと認証基盤

どこで認証して、何を持ち回るか。

SSO の3つの古典的な方式

リバースプロキシ方式

利用者

リバースプロキシ
ここで認証・中継

Web サーバ群
改修不要

全通信が集中 → **単一障害点**（R4秋・R6秋の正解）

エージェント方式

利用者

Web サーバ+エージェント
認証サーバと連携

認証サーバ
成功後は cookie

エージェントは **Web サーバに**入れる。

代理認証方式

利用者の PC+代行ソフト
ログイン画面を検知

ID・パスワードを代行入力

既存システム
改修不要

クライアント側で動く。**サーバの改修は不要。**

R6秋 問8 の誤りの選択肢は「エージェントがクライアント PC でログイン画面を監視」（＝代理認証の説明）、「代理認証は対象サーバの改修が必要」（逆）。

出典：IPA 午前 II 令和4年度秋期 問7・令和6年度秋期 問8

Kerberos：パスワードを流さずにチケットで

① 利用者 → AS：ログイン
パスワードから作った鍵で応答を復号できるか

② AS → 利用者：TGT
チケットをもらうためのチケット

③ 利用者 → TGS：TGT を示す
「ファイルサーバを使いたい」

④ TGS → 利用者：サービスチケット

⑤ 利用者 → ファイルサーバ：サービスチケットを示す

AS と TGS をまとめて **KDC**（AD ではドメインコントローラ）。

時刻のずれに弱い。チケットにタイムスタンプが入り、既定で5分程度を超えると失敗。

KDC の鍵（krbtgt）が盗まれると TGT を偽造できる＝**ゴールデンチケット**（#02）。

SSO の問題で「チケット」が出たら Kerberos、「アサーション」なら SAML。

出典：RFC 4120 The Kerberos Network Authentication Service (V5)／Microsoft Learn「Kerberos 認証の概要」

LDAP・IDaaS・SCIM：ID を管理する仕組み

LDAP（ディレクトリへの問い合わせ）

```
dn: uid=tanaka,ou=sales,dc=example,dc=jp
cn: 田中 一郎
mail: tanaka@example.jp
memberOf: cn=eigyo,ou=groups,...
```

LDAP	TCP 389。平文のバインドは盗聴される
LDAPS	TCP 636。TLS で保護（StartTLS も）

入力をフィルタに直結 → **LDAP インジェクション**（#03）。認証は「バインドできるか」で。

IDaaS を中心にした連携

IDaaS（Entra ID・Okta など）
認証・MFA・SSO・ログ

ログインのとき：SAML・OpenID Connect

アカウントの作成・削除：SCIM
REST＋JSON（RFC 7643・7644）

退職者を IDaaS で止めれば、SCIM で各 SaaS のアカウントも消える（**デプロビジョニング**）。

出典：RFC 4511・RFC 4513（LDAP）／RFC 7643・RFC 7644（SCIM 2.0）／IPA「支援士試験 シラバス（案）」用語例「LDAP サーバでの認証，IDaaS」

第2章

SAML によるアイデンティティ連携

認証は IdP、検証は SP。

SAML 認証の流れ（SP-initiated）

① 利用者が SP にアクセス → ② SP が認証要求を作り IdP へリダイレクト → ③ IdP が利用者認証（パスワード+MFA） → ④ IdP が署名したアサーションをブラウザ経由で SP へ → ⑤ SP が検証してログイン

SP が確かめること

署名	IdP の公開鍵で検証（改ざん・なりすまし）
Audience	自分宛てか（使い回し）
有効期限	古いものでないか（再利用）
InResponseTo	自分の要求への応答か

SP は**パスワードを受け取らない**。

XML 署名ラッピング：署名済みの要素を残したまま、偽の要素を差し込み、検証とアプリの読む場所のずれを突く。

出典：OASIS「Profiles for the OASIS SAML V2.0」（Web Browser SSO Profile）／IPA 午前 II 令和7年度春期 問2／OWASP「SAML Security Cheat Sheet」

SAMLと並べて出る規格を見分ける

規格	策定	何のためのものか
SAML	OASIS	組織（ドメイン）をまたいで**認証・属性・認可**の情報を XML で交換。SSO に使う
SOAP	W3C	XML でメッセージを交換する Web サービスの通信
XKMS	W3C	署名・暗号の**鍵情報**を Web サービスで管理
XML Signature	W3C	XML 文書へのデジタル署名の書式
UDDI	OASIS	Web サービスの情報を公開し、検索できるようにする
OpenID 2.0	OpenID Foundation	URL か XRI を利用者 ID にしてログイン（古い方式）

R4春 問2・R5秋 問8 は「SAML の説明」を、UDDI・S/MIME・XKMS・OAuth・OpenID・XML の説明の中から選ばせた。**OASIS／異なるドメイン／XML／認証・属性・認可**が SAML の目印。

出典：IPA 午前 II 平成31年度春期 問3・令和4年度春期 問2・令和5年度秋期 問8・令和6年度春期 問4

第3章

OAuth 2.0 と OpenID Connect

OAuth は「認可」。認証は OpenID Connect。

OAuth 2.0 の4つの役割（R6秋 問16）

リソースオーナー：利用者 A
「D に写真の読取りだけ許す」と同意

認可サーバ：Web サービス C
アクセストークンを発行

リソースサーバ：Web サービス C
リソース B を保管、トークンを確かめる

クライアント：Web サービス D
トークンを受け取り、B にアクセス

選択肢	判定
C がアクセストークンを発行	**正解**
D がアクセストークンを発行	**D は受け取る側**
A のデジタル証明書を送る	**OAuth にない**

R5秋・R7春：OAuth 2.0 は**認可**。認可サーバが、利用者の許可を得て、クライアントに権限を付与する。「本人であるかを確認する」は認証の説明で誤り。

出典：RFC 6749 The OAuth 2.0 Authorization Framework 1.1 Roles／IPA 午前 II 令和6年度秋期 問16・令和7年度春期 問16

認可コードグラントと PKCE

① クライアントが code_verifier を作り
S256 ハッシュ=code_challenge

→

② 認可要求に code_challenge を付ける
利用者が認可サーバで同意

→

③ 認可コードがリダイレクトで返る
ここで横取りされても…

→

④ コード+code_verifierでトークンと交換
ハッシュが一致しなければ拒否

グラント	RFC 9700（2025）
認可コード+PKCE	**推奨（全クライアントで PKCE）**
インプリシット（URL でトークン）	**使うべきでない**
パスワードをクライアントに渡す方式	**使ってはならない**
クライアントクレデンシャル	**サーバ同士（利用者なし）**

PKCE はもともと、秘密の鍵を安全にもてない**スマホアプリなどのパブリッククライアント**のために作られた（RFC 7636）。

出典：RFC 7636 Proof Key for Code Exchange／RFC 9700 OAuth 2.0 Security Best Current Practice 2.1.1・2.1.2・2.4

4つのトークンを見分ける

トークン	規格	だれがだれに見せる	何を表す・注意
認可コード	OAuth 2.0	クライアント → 認可サーバ	一回限りの引換券。PKCE で守る
アクセストークン	OAuth 2.0	クライアント → リソースサーバ	スコープと期限付きの許可。Bearer なので盗まれたら使われる → 短い期限
リフレッシュトークン	OAuth 2.0	クライアント → 認可サーバ	新しいアクセストークンをもらう。寿命が長い → ローテーション
ID トークン	OpenID Connect	OP（IdP） → RP	だれが・いつ・どう認証されたか。署名付き JWT

OpenID Connect ＝ OAuth 2.0 ＋ 認証の層。アクセストークンだけでログインさせると、別アプリ用のトークンで他人になりすませる。

出典：RFC 6749 1.4・1.5／RFC 6750 Bearer Token Usage／OpenID Connect Core 1.0 2 ID Token

ID トークンと JWT の検証

JWT = ヘッダ.ペイロード.署名（Base64URL）

```
{"alg":"RS256","kid":"k1"}
.
{"iss":"https://idp.example.jp",
 "sub":"u12345", "aud":"rp-app-01",
 "exp":1793500000, "iat":1793499700,
 "nonce":"n-0S6_WzA2Mj"}
.
（IdP の秘密鍵による署名）
```

JWS は改ざん検出、**中身は誰でも読める**。隠すなら JWE。

確かめる	確かめないと
署名と alg	alg "none" の偽トークンが通る
iss	別の発行者のトークンを信じる
aud	他の RP 宛てを使い回される
exp	期限切れが使える
nonce	リプレイ・トークンの差し込み

受け入れるアルゴリズムは**サーバ側で固定**（RFC 8725）。

出典：RFC 7519（JWT）・RFC 7515（JWS）・RFC 7516（JWE）・RFC 8725（JWT BCP）／OpenID Connect Core 1.0 3.1.3.7。JWT の例は Claude が作った

まとめ：本番での問われ方

SSO の方式：リバースプロキシ＝集中・単一障害点、エージェント＝Web サーバに導入、代理認証＝クライアントで代行入力・改修不要。

持ち回るもの：Kerberos＝チケット、SAML＝アサーション、OAuth＝アクセストークン、OIDC＝ID トークン。

SAML の目印：OASIS・異なるドメイン・XML・認証/属性/認可。認証は IdP、検証は SP。

OAuth 2.0 は認可：利用者の許可のもと、認可サーバがクライアントにアクセストークンを発行。本人確認は OpenID Connect。

守り方：PKCE（コードの横取り）、短いアクセストークン、リフレッシュトークンのローテーション、aud・alg の検証。

IDaaS＋SCIM で、アカウントの削除まで一元化。

出典：IPA 午前 II（このPDFの問題演習）／IPA「支援士試験 シラバス（案）Ver.0.1」